

Algoritmica Grafurilor - Cursul 7

13 novembre 2020

1

Cuplaje

- Formularea analitică a problemei cuplajului maxim
- Cuplaje perfecte - Teorema lui Tutte
- Cuplaje de cardinal maxim - Teorema lui Berge
- Cuplaje de cardinal maxim - Algoritmul Hopcroft-Karp

2

Fluxuri în rețele

- Rețele de transport
- Flux, valoarea fluxului, problema fluxului maxim

3

Exerciții pentru seminarul următor

4

Exerciții rezolvate (parțial)

Formularea analitică problemei cuplajului maxim

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Fie $G = (V, E)$ un graf și $\mathcal{M}_G$ familia cuplajelor sale.

Problem cuplajului maximum.

P_1 dat un graf $G = (V, E)$, determinați $M^* \in \mathcal{M}_G$ astfel încât

$$|M^*| = \max_{M \in \mathcal{M}_G} |M|.$$

Fie $B = (b_{ij})_{n \times m}$ matricea de incidență a lui G (se consideră câte o ordonare fixată a celor n noduri al lui G și a celor m muchii ale lui G) cu

$$b_{ij} = \begin{cases} 1, & \text{dacă muchia } j \text{ este incidentă cu nodul } i \\ 0, & \text{altfel} \end{cases}$$

Dacă 1_p este vectorul p -dimensional cu toate componentele 1, atunci problema P_1 poate fi descrisă echivalent astfel:

- Graph Algorithms - C. Croitoru - Graph Algorithms - C. Croitoru - Graph Algorithms

Formularea analitică a problemei cuplajului maxim

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

$$P'_1 \max \{1_m^T x : Bx \leq 1_n, x \geq 0, x_i \in \{0, 1\}, i = \overline{1, m}\}.$$

P'_1 este o problemă ILP (Integer Linear Programming), care este în general NP-hard. Vom încerca să rezolvăm P'_1 , folosind problema (relaxată) asociată LP (Linear Programming)

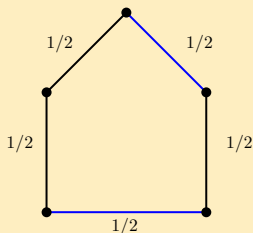
$$LP'_1 \max \{1_m^T x : Bx \leq 1_n, x \geq 0\}.$$

Soluțiile optime ale lui LP'_1 pot avea componente ne-întregi și de asemenea valorile lor optime pot fi mai mari decât $\nu(G)$. De exemplu, dacă $G = C_{2n+1}$, atunci $\nu(G) = n$, dar soluția $x_i = 1/2, \forall 1 \leq i \leq 2n+1$ este optimă pentru problema corespunzătoare LP'_1 cu valoarea optimă $n + 1/2 > n$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Formularea analitică a problemei cuplajului maxim

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms



Urmează că dacă graful G are circuite impare, problema P_1 nu poate fi rezolvată folosind LP'_1 . Mai precis,

Teorema 1

(Balinski, 1971) Punctele extreme ale politopului $Bx \leq 1_n, x \geq 0, x \in \mathbb{R}^m$, au componentele din $\{0, 1/2, 1\}$. Componentele $1/2$ apar dacă și numai dacă G are circuite impare.

Cuplaje perfecte - Teorema lui Tutte

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Teorema 2

(Hall, 1935) Fie $G = (S, T; E)$ un graf bipartit. Există un cuplaj în G care saturează toate nodurile din S dacă și numai dacă

$$|N_G(A)| \geq |A|, \forall A \subseteq S.$$

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Teorema 3

(Konig, 1930) Fie $G = (S, T; E)$ un graf bipartit. Cardinalul maxim al unui cuplaj în G este egal cu cardinalul minim al unei acoperiri cu noduri a lui G , $\nu(G) = n - \alpha(G)$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Fie G be un graf. Evident, $|M| \leq |G|/2, \forall M \in \mathcal{M}_G$. Un **cuplaj perfect** (sau **1-factor**) în G este un cuplaj M cu proprietatea $|M| = |G|/2$ (i.e., $S(M) = V(G)$).

O componentă conexă a grafului G este pară (impară) dacă numărul nodurilor sale este par (impar). **Notăm cu $q(G)$ numărul componentelor conexe impare ale lui G .**

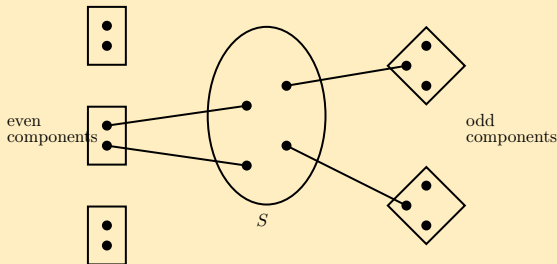
Remarcă

Fie G un graf care are un cuplaj perfect. Evident, fiecare componentă conexă a lui G este pară. Astfel, $q(G) = 0$. Mai mult, dacă $S \subseteq V(G)$ atunci pentru fiecare componentă conexă impară din graful $G - S$ trebuie să existe o muchie în cuplajul perfect al grafului cu o extremitate în această componentă conexă și cealaltă în S . Deoarece extremitățile muchiilor diferite sunt distincte, urmează că $|S| \geq q(G - S)$.

Pentru $S = \emptyset$ în (T), obținem $q(G - \emptyset) \leq 0$, deci $q(G) = 0$.

Cuplaje perfecte - Teorema lui Tutte

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph



Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Teorema 4

(Tutte, 1947) Un graf G admite cuplaj perfect dacă și numai dacă

$$(T) \quad q(G - S) \leq |S|, \forall S \subseteq V(G).$$

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Demonstrație. Necesitatea condiției (T) a fost dovedită în discuția de mai sus. Demonstrăm prin inducție după $n = |G|$ că dacă un graf $G = (V, E)$ satisface (T), atunci G are un cuplaj perfect.

Pentru $n = 1, 2$ teorema are loc evident. În pasul inductiv, fie G un graf cu $n \geq 3$ noduri care satisface (T) și să presupunem că orice graf G' cu $|G'| < n$ și care satisface (T) are un cuplaj perfect.

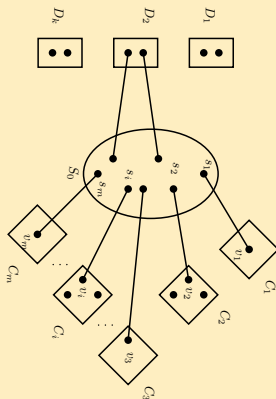
Fie $S_0 \subseteq V(G)$ astfel încât $q(G - S_0) = |S_0|$ și maximală cu proprietatea că avem egalitate în (T) (i. e., pentru orice supramulțime S a lui S_0 avem $q(G - S) < |S|$).

Observăm că familia de submulțimi ale lui $V(G)$ pentru care (T) este satisfăcută cu egalitate este nevidă, și, deci, există un S_0 (pentru fiecare nod v_0 care nu este punct de articulație în componența lui din G , avem $q(G - v_0) = 1 = |\{v_0\}|$, deoarece fiecare componentă conexă este pară și în fiecare componentă conexă cu cel puțin un nod, există un astfel de nod v_0).

Cuplaje perfecte - Teorema lui Tutte

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Fie $m = |S_0| > 0$, $C_1, C_2, \dots, C_m$ componentele impare ale lui $G - S_0$ și $D_1, D_2, \dots, D_k$ componentele pare ale lui $G - S_0$ ($k \geq 0$):



Vom construi un cuplaj perfect compus din

Cuplaje perfecte - Teorema lui Tutte

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- a) câte un cuplaj perfect în fiecare componentă conexă pară D_i ;
- b) un cuplaj cu m muchii, $\{e_1, \dots, e_m\}$, muchia e_i având un capăt $s_i \in S_0$ și celălalt $v_i \in C_i$ ($i = 1, \overline{1, m}$);
- c) cuplaj perfect în fiecare subgraph $C_i - v_i$ ($i = 1, \overline{1, m}$).

a) Pentru orice $1 \leq i \leq k$ graful $[D_i]_G$ admite cuplaj perfect. Într-adevăr, deoarece $m > 0$, urmează că $|D_i| < n$ și din ipoteza inductivă este suficient să arătăm că $G' = [D_i]_G$ satisface (T).

Fie $S' \subseteq D_i$. Dacă $q(G' - S') > |S'|$, atunci obținem următoarea contradicție:

$$q(G - (S_0 \cup S')) = q(G - S_0) + q(G' - S') = |S_0| + q(G' - S') > |S_0 \cup S'|.$$

Astfel, $q(G' - S') \leq |S'|$, $\forall S' \subseteq D_i$, i. e., G' satisface (T).

Cuplaje perfecte - Teorema lui Tutte

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

b) Fie $H = (S_0, \{C_1, \dots, C_m\}; E')$ graful bipartit cu o clasă a bipartiției S_0 , cealaltă clasă mulțimea componentelor conexe impare ale lui $G - S_0$, și cu muchiile de forma $\{s, C_i\}$, unde $s \in S_0$ astfel că există $v \in C_i$ cu $sv \in E(G)$.

H are un cuplaj perfect. Într-adevăr, arătăm că H satisface condiția din teorema lui Hall pentru existența unui cuplaj M_0 care saturează $\{C_1, \dots, C_m\}$.

Fie $A \subseteq \{C_1, \dots, C_m\}$. Atunci $B = N_H(A) \subseteq S_0$, și din construcția lui H , în graful G nu avem nicio muchie de la un nod $v \in S_0 - B$ la un nod $w \in C_i \in A$. Astfel componentele conexe impare din A rămân componente conexe impare și în $G - B$; astfel $q(G - B) \geq |A|$. Deoarece G satisface condiția lui Tutte (T), avem $|B| \geq q(G - B)$. Am obținut că $|N_H(A)| = |B| \geq |A|$.

Cuplaje perfecte - Teorema lui Tutte

Din teorema lui Hall H are un cuplaj M_0 , care saturează $\{C_1, \dots, C_m\}$; deoarece $|S_0| = m$, M_0 este perfect.

$$M_0 = \{s_1 v_1, s_2 v_2, \dots, s_m v_m\}, S_0 = \{s_1, \dots, s_m\}, v_i \in C_i, \forall i = \overline{1, m}.$$

c) $\forall i \in \{1, \dots, m\}$ graful $G' = [C_i - v_i]_G$ admite cuplaj perfect. Folosind ipoteza inductivă, este suficient să dovedim că G' satisface (T).

Fie $S' \subseteq C_i - v_i$. Dacă $q(G' - S') > |S'|$, atunci, deoarece $q(G' - S') + |S'| \equiv 0 \pmod{2}$ (pentru că $|G'|$ este par), urmează că $q(G' - S') \geq |S'| + 2$.

Dacă $S'' = S_0 \cup \{v_i\} \cup S'$, avem

$$\begin{aligned} |S''| &\geq q(G - S'') = q(G - S_0) - 1 + q(G' - S') = |S_0| - 1 + q(G' - S') \geq \\ &\geq |S_0| - 1 + |S'| + 2 = |S''|, \end{aligned}$$

i. e., $q(G - S'') = |S''|$, în contradicție cu alegerea lui S_0 (deoarece $S_0 \subsetneq S''$).

Cuplaje perfecte - Teorema lui Tutte

Astfel $\forall S' \subseteq C_i - v_i$, $q(G' - S') \leq |S'|$ și G' are cuplaj perfect (din ipoteza inductivă).

Evident cuplajul lui G obținut reunind cuplajele de la a), b), și c) de mai sus saturează toate nodurile lui G , și demonstrația prin inducție a teoremei se încheie

Cuplaje de cardinal maxim - Teorema lui Berge

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Fie $G = (V, E)$ un graf și $M \in \mathcal{M}_G$ un cuplaj al lui G .

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Definiție

Un drum alternat în G relativ la cuplajul M este un drum

$$P : v_0, v_0 v_1, v_1, \dots, v_{k-1}, v_{k-1} v_k, v_k$$

astfel încât $\{v_{i-1} v_i, v_i v_{i+1}\} \cap M \neq \emptyset, \forall i = \overline{1, k-1}$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

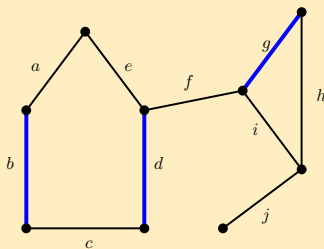
Observăm că, deoarece M este un cuplaj, dacă P este un drum alternat relativ la M , atunci dintre orice două muchii consecutive ale lui P exact una aparține lui M (muchiiile aparțin alternativ lui M și $E \setminus M$).

În cele ce urmează, când ne vom referi la un drum P vom înțelege mulțimea muchiilor sale.

Definiție

Un drum de creștere al lui G relativ la cuplajul M este un drum alternat care unește două noduri distincte expuse relativ la M .

Observăm că, din definiția de mai sus, urmează că dacă P este un drum de creștere relativ la M , atunci $|P \setminus M| = |P \cap M| + 1$.



a, b, c, d - alternating even path

f - alternating odd path

j - augmenting path

g, f, d - alternating odd path

a, b, c, d, e - closed alternating path

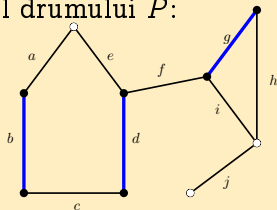
a, b, c, d, f, g, h - augmenting path

Teorema 5

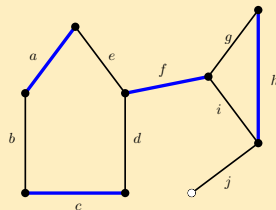
(Berge, 1959) M este un cuplaj de cardinal maxim în graful G dacă și numai dacă nu există drum de creștere în G relativ la M .

Demonstrație. " $\Rightarrow$ " Fie M un cuplaj de cardinal maxim în G . Să presupunem că P este un drum de creștere în G relativ la M .

Atunci, $M' = M \Delta P = (P \setminus M) \cup (M \setminus P) \in \mathcal{M}_G$. Într-adevăr, M' poate fi obținut prin interschimbarea muchiilor din M cu cele din afara lui M de-a lungul drumului P :



$P = a, b, c, d, f, g, h$ - augmenting path



$M \Delta P$

Cuplaje de cardinal maxim - Teorema lui Berge

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Mai mult, $|M'| = |P \cap M| + 1 + |M \setminus P| = |M| + 1$, în contradicție cu alegerea lui M .

" $\Leftarrow$ " Fie M un cuplaj în G cu proprietatea că nu există drumuri de creștere în G relativ la M .

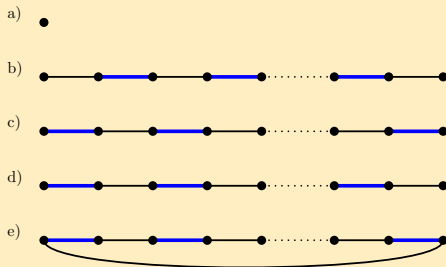
Dacă M^* este un cuplaj de cardinal maxim în G , vom arăta că $|M^*| = |M|$. Fie G' subgraful generat de $M \Delta M^*$ în G ($G' = (V, M \Delta M^*)$).

Să observăm că $d_{G'}(v) \leq 2$, $\forall v \in V$ și deci componentele conexe ale lui G' pot fi noduri izolate, drumuri de lungime cel puțin unu, sau circuite. Avem cinci posibilități (se văd mai jos; muchiile albastre sunt din M^* , muchiile negre sunt din M).

Cazul b) nu apare deoarece este un drum de creștere relativ la M^* , care este un cuplaj de cardinal maxim. Cazul c) nu apare deoarece este un drum de creștere relativ la M .

- Graph Algorithms - C. Croitoru - Graph Algorithms - C. Croitoru - Graph Algorithms

Cuplaje de cardinal maxim - Teorema lui Berge



Dacă notăm cu $m_M(C)$ numărul de muchii din M din componenta conexă C a lui G' și cu $m_{M^*}(C)$ numărul de muchii din M^* din aceeași componentă conexă a lui G' , obținem că $m_M(C) = m_{M^*}(C)$. Astfel,

$$\begin{aligned} |M \setminus M^*| &= \sum_{C \text{ comp. a lui } G'} m_M(C) = \\ &= \sum_{C \text{ comp. a lui } G'} m_{M^*}(C) = |M^* \setminus M| \end{aligned}$$

Cuplaje de cardinal maxim - Teorema lui Berge

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Deci $|M| = |M^*|$. $\square$

Obținem o strategie pentru a determina un cuplaj de cardinal maxim:

```
fie  $M$  un cuplaj în  $G$  (e. g.,  $M = \emptyset$ );  
while ( $\exists P$  drum de creștere relativ la  $M$ ) do  
     $M \leftarrow M \Delta P$ ;  
end while
```

La fiecare iterație **while** cardinalul lui M crește cu 1, astfel, în cel mult $n/2$ iterații obținem un cuplaj fără drumuri de creștere, adică de cardinal maxim. Neajunsul acestui algoritm este următorul: condiția din **while** trebuie implementată în timp polinomial. Acest lucru a fost făcut pentru prima oară de către **Edmonds (1965)**.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Cuplaje de cardinal maxim - Algoritmul Hopcroft-Karp

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Lema 1

Fie $M, N \in \mathcal{M}_G$, $|M| = r$, $|N| = s$ și $s > r$. Atunci în $M \Delta N$ există cel puțin $s - r$ drumuri de creștere relativ la M disjuncte pe noduri.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Demonstrație. Fie $G' = (V, M \Delta N)$ și C_i ($i = \overline{1, p}$) componentele conexe ale lui G' . Pentru fiecare $1 \leq i \leq p$, notăm cu $\delta(C_i)$ diferența dintre numărul de muchii ale lui N din C_i și numărul de muchii ale lui M din C_i :

$$\delta(C_i) = |E(C_i) \cap N| - |E(C_i) \cap M|.$$

Se observă că, deoarece M și N sunt cuplaje, C_i sunt drumuri sau circuite. Astfel $\delta(C_i) \in \{-1, 0, 1\}$. $\delta(C_i) = 1$ dacă și numai dacă C_i este un drum de creștere relativ la M .

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Demonstrație (continuare). Deoarece

$$\sum_{i=1}^p \delta(C_i) = |N \setminus M| - |M \setminus N| = s - r,$$

urmează că există cel puțin $s - r$ componente conexe ale lui G' cu $\delta(C_i) = 1$, adică, există cel puțin $s - r$ drumuri de creștere disjuncte pe noduri conținute în $M \Delta N$. $\square$

Lema 2

Dacă $\nu(G) = s$ și $M \in \mathcal{M}_G$ cu $|M| = r < s$, atunci există în G un drum de creștere relativ la M de lungime cel mult $2\lceil r/(s - r) \rceil + 1$.

Demonstrație. Fie $N \in \mathcal{M}_G$ cu $|N| = s = \nu(G)$.

Cuplaje de cardinal maxim - Algoritmul Hopcroft-Karp

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Demonstrație (continuare). Din Lema 1, există $s - r$ drumuri de creștere disjuncte pe muchii (drumurile disjuncte pe noduri sunt și disjuncte pe muchii) conținute în $M \Delta N$. Urmează că cel puțin unul dintre ele are cel mult $\lceil r/(s - r) \rceil$ muchii din M . Lungimea acestui drum de creștere este cel mult $2\lceil r/(s - r) \rceil + 1$. $\square$

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Definiție

Fie $M \in \mathcal{M}_G$. Un **drum de creștere minim** relativ la M în G este un drum de creștere de lungime minimă printre toate drumurile de creștere relativ la M din G .

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Lema 3

Fie $M \in \mathcal{M}_G$, P un drum de creștere minim relativ la M , și P' un drum de creștere relativ la $M \Delta P$. Atunci $|P'| \geq |P| + 2|P \cap P'|$.

Demonstrație. Fie $N = (M \Delta P) \Delta P'$. Atunci $M \Delta N = P \Delta P'$ și $|N| = |M| + 2$. Din Lema 1, există două drumuri de creștere relativ la M disjuncte pe muchii, P_1 și P_2 , conținute în $M \Delta N$. Deoarece P este un drum de creștere minim relativ la M , avem $|P \Delta P'| \geq |P_1| + |P_2| \geq 2|P|$ și, deci, $|P| + |P'| - 2|P \cap P'| \geq 2|P|$. $\square$

Considerăm următorul algoritm:

```
 $M_0 \leftarrow \emptyset; i = 0;$   
while ( $\exists$  drumuri de creștere relativ la  $M$ ) do  
    fie  $P_i$  un drum de creștere minim relativ la  $M_i$ ;  
     $M_{i+1} \leftarrow M_i \Delta P_i$ ;  
     $i++$ ;  
end while
```

Fie $P_0, P_1, \dots, P_{\nu(G)-1}$ secvența de drumuri de creștere minime construite de acest algoritm.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Lema 4

- a) $|P_i| \leq |P_{i+1}|$ și $|P_i| = |P_{i+1}|$ dacă și numai dacă P_i și P_{i+1} sunt disjuncte pe noduri, $\forall 1 \leq i \leq \nu(G) - 2$.
- b) $\forall i < j < \nu(G) - 1$, dacă $|P_i| = |P_j|$, atunci P_i și P_j sunt disjuncte pe noduri.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

- Demonstrație.** a) Luând $P = P_i$ și $P' = P_{i+1}$ în Lema 3, obținem $|P_{i+1}| \geq |P_i| + 2|P_i \cap P_{i+1}| \geq |P_i|$. Egalitatea are loc dacă și numai dacă P_i și P_{i+1} sunt disjuncte pe muchii, de unde rezultă că sunt disjuncte și pe noduri (fiind drumuri alternate).
- b) rezultă aplicând succesiv a) $\square$

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Teorema 6

(Hopcroft, Karp, 1973) Fie G un graf cu $\nu(G) = s$. Numărul de întregi distincți din secvența $|P_0|, |P_1|, \dots, |P_{s-1}|$ (P_i sunt drumuri de creștere minime construite de algoritmul de mai sus) nu este mai mare decât $2\lfloor\sqrt{s}\rfloor + 2$.

Demonstrație. Fie $r = \lceil s - \sqrt{s} \rceil$. Atunci $|M_r| = r$ și

$$|P_r| \leq 2\lceil r/(s-r) \rceil + 1 = 2\lceil \lceil s - \sqrt{s} \rceil / (s - \lceil s - \sqrt{s} \rceil) \rceil + 1 < 2\lfloor\sqrt{s}\rfloor + 3.$$

Astfel, pentru orice $i < r$, $|P_i|$ este unul din cele $\lfloor\sqrt{s}\rfloor + 1$ numere întregi impare nu mai mari decât $2\lfloor\sqrt{s}\rfloor + 1$.

În sub-secvența $|P_r|, \dots, |P_{s-1}|$ există cel mult $s - r \leq \lfloor\sqrt{s}\rfloor + 1$ întregi distincți. Urmează că în secvența $|P_0|, |P_1|, \dots, |P_{s-1}|$ nu există mai mult de $2\lfloor\sqrt{s}\rfloor + 2$ întregi distincți. $\square$

$$2\lceil [s - \sqrt{s}] / (s - \lceil s - \sqrt{s} \rceil) \rceil + 1 < 2\lfloor \sqrt{s} \rfloor + 3 \Leftrightarrow$$

$$2\lceil [s - \sqrt{s}] / (s - \lceil s - \sqrt{s} \rceil) \rceil < 2\lfloor \sqrt{s} \rfloor + 2 \Leftrightarrow$$

$$\lceil [s - \sqrt{s}] / (s - \lceil s - \sqrt{s} \rceil) \rceil \leq \lfloor \sqrt{s} \rfloor + 1 \Leftrightarrow$$

$$(s - \lfloor \sqrt{s} \rfloor) / \lfloor \sqrt{s} \rfloor \leq \lfloor \sqrt{s} \rfloor + 1 \Leftrightarrow s \leq \lfloor \sqrt{s} \rfloor^2 + 2\lfloor \sqrt{s} \rfloor$$

If $\lfloor \sqrt{s} \rfloor = k \in \mathbb{N}$, then $k \leq \sqrt{s} < k + 1$, deci $s = (\sqrt{s})^2 < k^2 + 2k + 1$ adică $s = (\sqrt{s})^2 \leq k^2 + 2k$ - care este inegalitatea de mai sus.

Dacă algoritmul de mai sus este descompus în faze astfel încât în fiecare fază se determină o familie maximală de drumuri de creștere minime disjuncte pe noduri , atunci - din Lema 4 - lungimea drumurilor de creștere minime din faza următoare nu va descrește (altfel, mulțimea drumurilor de creștere minime construite în faza curentă nu este maximală). Din Teorema 6, obținem că numărul de faze nu este mai mare de $2\lfloor \sqrt{\nu(G)} \rfloor + 2$.

Astfel avem următorul algoritm pentru determinarea unui cuplaj de cardinal maxim într-un graf dat G :

```
 $M \leftarrow \emptyset;$   
repeat  
    determină  $\mathcal{P}$  o familie maximală de drumuri de creștere minime  
    relativ la  $M$  disjuncte pe noduri;  
    for ( $P \in \mathcal{P}$ ) do  
         $M \leftarrow M \Delta P;$   
    end for  
until ( $\mathcal{P} = \emptyset$ )
```

Complexitatea timp a algoritmului de mai sus este $\mathcal{O}(\sqrt{n}A)$, unde A este complexitatea timp a determinării familiei $\mathcal{P}$.

În cazul grafurilor bipartite, **Hopcroft** și **Karp** au arătat că aceasta poate fi obținută în $\mathcal{O}(n+m)$ și, deci, întreg algoritmul are complexitatea timp $\mathcal{O}(m\sqrt{n})$.

Cuplaje de cardinal maxim - Algoritmul Hopcroft-Karp

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

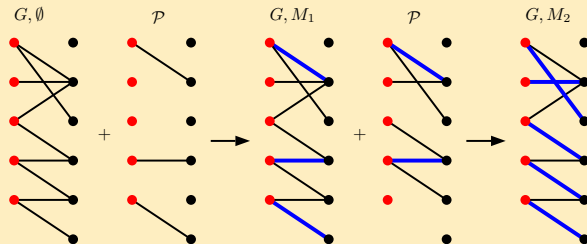
Acest rezultat a fost extins la grafuri arbitrare de către **Micali** și **Vazirani (1980)** folosind o structură de date elaborată pentru a întreține etichetele asociate nodurilor pentru construcția drumurilor de creștere minime.

Să considerăm cazul grafurilor bipartite: $G = (S, T; E)$ și $M \in \mathcal{M}_G$. Pornind cu una dintre clase, de exemplu S , considerăm mulțimea extremităților inițiale ale unor drumuri de creștere $S \cap E(M)$. Din fiecare astfel de nod pornim, în paralel, construcția unor drumuri alternate într-o manieră **bfs**. Primul drum de creștere obținut oprește construcția, oferind lungimea minimă a unui drum de creștere. Familia $\mathcal{P}$ este obținută folosind etichetele și listele de adiacență în $\mathcal{O}(n + m)$.

Detaliile sunt omise; un exemplu este dat pe slide-ul următor.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Cuplaje de cardinal maxim - Algoritmul Hopcroft-Karp



Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

O rețea (de transport) cu sursa s și destinația t este o tuplă $R = (G, s, t, c)$ unde:

- $G = (V, E)$ este un digraf,
- $s, t \in V$; $s \neq t$; $d_G^+(s) > 0$; $d_G^-(t) > 0$,
- $c : E \rightarrow \mathbb{R}_+$; $c(e)$ este capacitatea arcului e .

Vom presupune că $V = \{1, 2, \dots, n\}$ ($n \in \mathbb{N}^*$) și $|E| = m$. Extindem funcția c la $c : V \times V \rightarrow \mathbb{R}_+$ prin

$$c((i, j)) = \begin{cases} c(ij), & \text{dacă } ij \in E \\ 0, & \text{altfel} \end{cases}$$

și notăm $c((i, j)) = c_{ij}$.

Definiție

Un **flux** în $R = (G, s, t, c)$ este o funcție $x : V \times V \rightarrow \mathbb{R}$ a. î.

- (i) $0 \leq x_{ij} \leq c_{ij}, \forall (i, j) \in V \times V,$
- (ii) $\sum_{j \in V} x_{ji} - \sum_{j \in V} x_{ij} = 0, \forall i \in V \setminus \{s, t\}.$

Remarci

- Dacă $ij \in E$ atunci x_{ij} este **flux** (transportat) de-a lungul arcului ij .
- Constrângerile (i) cer ca **fluxul** pe fiecare arc să fie ne-negativ și să nu depășească capacitatea.
- Constrângerile (ii), (**legea de conservare**), cer ca **suma fluxurilor** de pe arcele care intră într-un nod i să fie egală cu suma fluxurilor de pe arcele care ies din i (fluxul care intră este egal cu fluxul care iese).

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- Să observăm că, din modul în care am extins funcția c la $V \times V$, constrângerile (i) implică $x_{ij} = 0$ când $ij \notin E$. Astfel, un flux este de fapt o funcție definită pe E . Preferăm extensia sa pe $V \times V$ pentru a simplifica notațiile.

Fie x un flux în $R = (G, s, t, c)$. Dacă adunăm toate constrângerile (ii) (pentru $i \in V \setminus \{s, t\}$) obținem

$$\begin{aligned} 0 &= \sum_{i \neq s, t} \left(\sum_{j \in V} x_{ji} - \sum_{j \in V} x_{ij} \right) = \sum_{i \neq s, t} \sum_{j \neq s, t} x_{ji} - \sum_{i \neq s, t} \sum_{j \neq s, t} x_{ij} + \\ &\quad + \sum_{i \neq s, t} x_{si} + \sum_{i \neq s, t} x_{ti} - \sum_{i \neq s, t} x_{is} - \sum_{i \neq s, t} x_{it} = \end{aligned}$$

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

$$= \left(\sum_{i \in V} x_{si} - \sum_{i \in V} x_{is} \right) - \left(\sum_{i \in V} x_{it} - \sum_{i \in V} x_{ti} \right),$$

Definiție

Valoarea fluxului x în $R = (G, s, t, c)$ este

$$v(x) = \sum_{i \in V} x_{it} - \sum_{i \in V} x_{ti}.$$

În cuvinte, $v(x)$ este **fluxul net care ajunge în destinația** rețelei sau, după cum am demonstrat mai sus, **fluxul net care părăsește sursa** rețelei.

Să observăm că în orice rețea $R = (G, s, t, c)$ există un flux: x^0 , **fluxul nul**, cu $x_{ij}^0 = 0, \forall ij \in E$ și $v(x^0) = 0$.

Problema fluxului maxim: Dată $R = (G, s, t, c)$ o rețea, să se determine un flux de valoare maximă.

Problema fluxului maxim poate fi văzută ca o problemă LP:

$$\begin{aligned} \max \quad & v \\ & \sum_{j \in V} x_{ji} - \sum_{j \in V} x_{ij} = 0, \forall i \neq s, t \\ & \sum_{j \in V} x_{js} - \sum_{j \in V} x_{sj} = -v \\ & \sum_{j \in V} x_{jt} - \sum_{j \in V} x_{tj} = v \\ & 0 \leq x_{ij} \leq c_{ij}, \forall ij \in E \end{aligned}$$

Cu toate acestea, vom considera a abordare combinatorială directă care este importantă când există constrângeri de integralitate a unor variabile (de exemplu fluxul de pe arce).

Exercițiul 1. Fie X o mulțime finită, $X_1, \dots, X_n \subseteq X$, și $d_1, d_2, \dots, d_n \in \mathbb{N}$. Arătați că există n submulțimi disjuncte $Y_i \subseteq X_i$, $|Y_i| = d_i$, $\forall i = \overline{1, n}$ dacă și numai dacă

$$\left| \bigcup_{i \in I} X_i \right| \geq \sum_{i \in I} d_i,$$

pentru orice $I \subseteq \{1, \dots, n\}$.

Exercițiul 2. Orice graf p -regulat bipartit admite cuplaj perfect.

Exercițiul 3. Fie $G = (S, T; E)$ un graf bipartit. Folosind teorema lui Hall demonstrați că, pentru fiecare $0 \leq k \leq |S|$, G are un cuplaj de cardinal cel puțin $|S| - k$ dacă și numai dacă $|N_G(A)| \geq |A| - k$, $\forall A \subseteq S$.

Exercițiul 4. Utilizând teorema lui Tutte arătați că un graf 2-muchie conex și 3-regulat admite cuplaj perfect.

Exercițiul 5. Fie $G = (V, E)$ un graf. O submulțime $A \subseteq V$ este numită *m-independentă* dacă G are un cuplaj M care saturează toate nodurile din A . Arătați că, pentru orice două mulțimi *m-independente* A și B cu $|A| < |B|$, se poate determina un nod $b \in B \setminus A$ astfel încât $A \cup \{b\}$ este de asemenea *m-independentă* ($\Rightarrow$ toate mulțimile maximal *m-independente* au același cardinal).

Exercițiul 6. Fie $T = (V, E)$ un arbore cu rădăcină; îi notăm cu r rădăcina și cu $\text{parent}(v)$ strămoșul direct al oricărui nod $v \neq r$. Un cuplaj M al lui T este numit *propriu* dacă orice nod nesaturat de M , $v \neq r$, are un frate w astfel încât $w \text{parent}(v) \in M$.

- (a) Arătați că orice cuplaj *propriu* este un cuplaj de cardinal maxim.
- (b) Găsiți în $\mathcal{O}(n)$ un cuplaj *propriu* pentru un arbore de ordin n .

Exercițiul 7. Fie $G = (V, E)$ un graf p -regulat bipartit. Considerăm următorul algoritm:

```
for ( $e \in E$ ) do
     $a(e) \leftarrow 1$ ;
end for
 $E^+ \leftarrow \{e \in E : a(e) > 0\}$ ;
while ( $G^+ = (V, E^+)$  conține un circuit  $C$ ) do
    fie  $C = M_1 \cup M_2$ , unde  $M_1$  și  $M_2$  sunt cuplaje a. î.  $a(M_1) \geq a(M_2)$ ; //
    pentru orice  $F \subseteq E$ ,  $a(F) = \sum_{e \in F} a(e)$ ;
    for ( $e \in E(C)$ ) do
        if ( $e \in M_1$ ) then
             $a(e) ++$ ;
        else
             $a(e) --$ ;
        end if
    end for
     $E^+ \leftarrow \{e \in E : a(e) > 0\}$ ;
end while
return  $E^+$ ;
```

Exercițiul 7. (cont.) Fie $f(E^+) = \sum a^2(e)$. Arătați că

- (a) după fiecare iterație **while** $f(E^+)$ este un număr întreg care crește cu cel puțin $|C|$ față de valoarea anterioară;
- (b) după fiecare iterație **while**, $\sum_{uv \in E^+} a(uv) = p, \forall u \in V$;
- (c) cât timp există muchii e cu $0 < a(e) < p$, algoritmul continuă; la final $a(e) = p, \forall e \in E^+$ și în E^+ se găsesc muchiile unui cuplaj perfect al lui G ;
- (d) numărul de iterații **while** este finit, la final $f(E^+) = np^2/2 = pm$, iar suma lungimilor tuturor circuitelor procesate este cel mult pm ;
- (e) toate circuitele pot fi găsite în complexitatea timp $\mathcal{O}(\sum_{C \text{ cycle}} |C|)$ folosind parcurgeri **dfs**;
- (f) complexitatea timp a algoritmului în ansamblu este $\mathcal{O}(pm)$.

Exerciții pentru seminarul următor

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 8. Fie $G = (S, T; E)$ un graf bipartit nenul. Arătați că următoarele afirmații sunt echivalente:

- (i) $G - \{x, y\}$ are un cuplaj perfect, $\forall x \in S, \forall y \in T$.
- (ii) G este conex și orice muchie a lui G aparține unui cuplaj perfect.
- (iii) $|S| = |T|$ și $\emptyset \neq A \subsetneq S$, $|N_G(A)| > |A|$.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exercițiul 9. Fie $G = (V, E)$ un graf conex care are un cuplaj perfect. Descrieți (și demonstrați corectitudinea) unui algoritm de complexitate timp $\mathcal{O}(|V| + |E|)$ care să construiască un arbore parțial T al lui G astfel ca $V(T)$ să admită o bipartiție în două mulțimi stabile de cardinal maxim ale lui T .

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 1. Soluție.

- Fie $G = (S, X, E)$ următorul graf bipartit: S este compusă din câte d_i copii ale submulțimilor X_i , $\forall i = \overline{1, n}$; pentru fiecare $x \in X$ și pentru fiecare $X'_i \in S$ (o copie a lui X_i) adăugăm xx'_i la E dacă și numai dacă $x \in X_i$.
- Există mulțimile cerute $(Y_i)_{1 \leq i \leq n}$ dacă și numai dacă există un cuplaj care saturează S (**puteți explica de ce?**), ceea ce, din teorema lui Hall, este echivalent cu

$$\forall A \subseteq S, |A| \leq |N_G(A)| \iff$$

$$\iff \forall (d'_i)_{1 \leq i \leq n} \subseteq \mathbb{N}, d'_i \leq d_i, \forall i = \overline{1, n} \text{ and}$$

$$\sum_{i=1}^n d'_i \leq \left| \bigcup_{i=\overline{1, n}, d'_i \neq 0} X_i \right|.$$

Luând $I = \{i : d'_i \neq 0\}$ obținem inegalitatea dorită.

Exercițiul 2. Soluție. Fie $G = (S, T; E)$ un graf bipartit și p -regulat.

- Este ușor de arătat că $|S| = |T|$ (cum?).
- Astfel, va fi suficient de demonstrat că există un cuplaj care saturează S .
- Fie $A \subseteq S$; există $k|A|$ muchii între A și $N_G(A)$, deci $k|A| \leq k|N_G(A)|$ care este numărul total de muchii care pleacă din $N_G(A)$.
- Obținem $|A| \leq |N_G(A)|$ și concluzia dorită din teorema lui Hall.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 3. Soluție.

- Fie $G' = (S, T \cup W; E')$ graful obținut astfel: adăugăm k noduri noi la T ($W = \{x_1, x_2, \dots, x_k\}$); toate aceste noduri vor fi adiacente cu toate nodurile din S : $E' = E \cup \{ux_i : u \in S, 1 \leq i \leq k\}$.
- Evident că G conține un cuplaj de cardinal cel puțin $|S| - k$ dacă și numai dacă G' conține un cuplaj care saturează toate nodurile din S (de ce?).
- În G' există un cuplaj M' , cu $|M'| = |S|$ dacă și numai dacă (de ce?):

$$|N_{G'}(A)| \geq |A|, \forall A \subseteq S$$

ceea ce este echivalent cu

$$|N_G(A)| + k \geq |A|, \forall A \subseteq S,$$

deoarece $N_{G'}(A) = N_G(A) \cup W$ și $|N_{G'}(A)| = |N_G(A)| + k$.

Exercițiul 4. Soluție. Fie G un graf 2-muchie conex și 3-regulat

- Vom arăta că $q(G - S) \leq |S|, \forall S \subseteq V$.
- G este de ordin par (**de ce?**); putem presupune că $S \neq \emptyset$.
- Fie H_i o componentă conexă impară a lui $G - S$; cum G este conex, există o muchie $e_1 = u_1 v_1$ cu $u_1 \in S$ și $v_1 \in V(H_i)$.
- $G - e_1$ este deasemenea conex (**de ce?**), deci există și o altă muchie $e_2 = u_2 v_2 \neq e_1$, cu $u_2 \in S, v_2 \in V(H_i)$.
- Să presupunem prin reducere la absurd că e_1 and e_2 sun singurele muchii care leagă S de H_i , atunci:
$$2|E(H_i)| = \sum_{v \in V(H_i)} d_{H_i}(v) \stackrel{\text{de ce?}}{=} 3|H_i| - 2 \equiv 1 \pmod{2}, \text{ contradicție.}$$
- Astfel, fiecare componentă conexă impară a lui $G - S$ este legată prin cel puțin trei muchii de S :

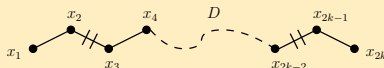
$$3|S| = \sum_{x \in S} d_G(x) \geq 3q(G - S).$$

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 6. Soluție.

- (a) Fie M un cuplaj propriu și să presupunem prin reducere la absurd că $|M| < \nu(T)$.
- Din teorema lui Berge T conține un M -drum de creștere D .
 - Un astfel de drum are lungime impară ≥ 3 (de ce?).
 - Presupunem că $V(D) = \{x_1, x_2, \dots, x_{2k}\}$, $x_1, x_{2k} \in E(M)$ și $x_2 = \text{parent}(x_1)$, $x_3 = \text{parent}(x_2)$.
 - Cum $x_1 \in S(M)$, există y un fată al lui x_1 cu $x_2y \in M$ - contradicție (de ce?).



Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

- (b) Folosim o parcurgere în adâncime (**de ce?** a arborelui pentru a construi un cuplaj propriu:

$$M \leftarrow \emptyset;$$
$$E(M) \leftarrow V(T);$$
$$ProperMatching(r);$$

- unde *ProperMatching* este descrisă recursiv mai jos:

```
ProperMatching(v); {  
  for( $x \in \mathcal{A}(v)$ ) {  
    ProperMatching(x);  
    if ( $v, x \in E(M)$ ) {  
       $M \leftarrow M \cup \{vx\};$   
       $E(M) \leftarrow E(M) \setminus \{x, v\};$   
    }  
  }  
}
```

Exercițiul 7. Soluție.

(a) Inițial $f(E^+) = m \in \mathbb{Z}_+$; după o iterație **while** $f(E^+)$ crește cu:

$$\sum_{e \in M_1} [(a(e) + 1)^2 - a^2(e)] + \sum_{e \in M_2} [(a(e) - 1)^2 - a^2(e)] =$$

$$\sum_{e \in M_1} [2a(e) + 1] \sum_{e \in M_2} [-2a(e) + 1] =$$

$$= 2[a(M_1) - a(M_2)] + |C| \geq |C|, a(M_i) \in \mathbb{Z}.$$

(b) Evident. **De ce?**

(c) Având o muchie e_1 cu $0 < a(e_1) < p$ putem aplica următoarea procedură: presupunem că $e_1 = u_1 u_2 \in E^+$; există o muchie $e_2 = u_2 u_3 \in E^+$ cu $0 < a(e_2) < p$ (**de ce?**), și tot așa până când atingem un nod, u_i , deja vizitat și se închide un circuit numai cu muchii din E^+ .

Exerciții rezolvate (parțial)

(d) La sfârșitul execuției algoritmului $|E^+| = n/2$ și $f(E^+) = np^2/2 = pm$ (de ce?). În fiecare pas $f(E^+)$ crește cu cel puțin $|C|$ deci creșterea totală a lui $f(E^+)$ este $(pm - m)$ iar suma lungimilor tuturor circuitelor este cel mult pm .

(e)

```
for  $v \in V$  do
     $parent(u) \leftarrow -1$ ;
end for
fie  $e = uv \in E^+$  cu  $0 < a(e) < p$ ;  $parent(u) \leftarrow 0$ ;  $parent(v) \leftarrow u$ ;
crează o stivă  $S$  conținând  $v$ ;
while  $S \neq \emptyset$  do
     $x \leftarrow top(S)$ ;
    if  $((y \leftarrow next[A(u)]) \neq NULL)$  then
        if  $parent(y) < 0$  then
             $parent(v) \leftarrow u$ ;  $push(S, v)$ ;
        else
            return  $y$ ;
        end if
    else
        delete( $S, x$ );
    end if
end while
```

- Folosind y returnat și tabloul $parent(\cdot)$ pute determina circuitul dorit. Nu există nicio muchie $e \in E^+$ cu $0 < a(e) < p$, deci $G^+ = (V, E^+)$ este aciclic (de ce?).

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 8. Soluție.

- (i) $\Rightarrow$ (ii) Dacă G nu este conex fie G' una dintre componentele sale conexe, $0 < |V(G') \cap S| \leq |V(G') \cap T|$. Luăm $u \in V(G') \cap S$, $v \in T \setminus V(G')$ și M un cuplaj perfect în $G - \{u, v\}$; $M \cap E(G')$ nu poate satura toate nodurile din G' (de ce?).
- (ii) $\Rightarrow$ (iii) Cum G are un cuplaj perfect, $|S| = |T|$ (de ce?). Din teorema lui Hall $|N_G(A)| \geq |A|$.
- Să presupunem prin reducere la absurd că $|N_G(A)| = |A|$, pentru o mulțime $\emptyset \neq A \subsetneq S$.
- $S \cup T \neq A \cup N_G(A)$ (de ce?), deci există două noduri adacente $u \in N_G(A)$ și $v \in S \setminus A$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

- Fie M be a cuplaj perfect care conține uv ; M conține deasemeni $|A|$ muchii de la A la $N_G(A)$, deci v este saturat în M de o muchie diferită de uv (de ce?) - contradicție.
- (iii) $\Rightarrow$ (i) Folosim teorema lui Hall pentru graful bipartit $G - \{x, y\}$. Pentru orice $A \subset S - \{x\}$

$$|N_{G-\{x,y\}}(A)| \geq |N_G(A)| - 1 \geq |A|.$$

Exercițiul 9. Soluție.

- Fie M un cuplaj perfect în $G = (V, E)$.
- Mai întâi reordonăm listele de adiacență astfel ca muchiile din M să apară la început. Fie următoarea procedură **dfs**

```
dfs( $v, i$ )  
   $visited[v] \leftarrow i$ ;  
  for ( $u \in \mathcal{A}(v)$ ) do  
    if ( $visited[u] = -1$ ) then  
       $E' \leftarrow E' \cup \{uv\}$ ; dfs( $u, 1 - i$ );  
    end if  
  end for
```

- Algoritmul principal este:

```
for ( $v \in V$ ) do
     $visited[v] \leftarrow -1$ ;
end for
let  $v_0 \in V$ ;
dfs( $v_0, 0$ );
```

- Definim $A = \{v \in V : visited[v] = 0\}$ și $B = \{v \in V : visited[v] = 1\}$; A și B sunt mulțimi stabile (în $T = (V, E')$) de același cardinal (de ce?).
- (Cu alte cuvinte, orice drum de la rădăcină la frunze este alternat: unul dintre ele începe și se termină cu câte o muchie din M iar celelalte încep cu câte o muchie din afara lui M și se încheie cu o muchie din M .)